30-Day Learning Journey · Tools · Concepts · Getting Started

Building with AI: What I Learned

A Sales Engineer's Field Guide to Building Real Tools with Claude Code

Version 1.0 · March 2026 · Sophos Sales Engineering · Internal Use Only

CHAPTERS

→ About This Guide

→ Chapter 1: Why This Guide Exists
Projects & What You'll Build

→ Chapter 2: The Stack
Tools, Costs, and Tiers

→ Chapter 3: Key Concepts Glossary
Plain-English Definitions

→ Chapter 4: Plan-Then-Execute Pattern
The Core Workflow

→ Chapter 5: Dos and Don'ts
Security-Aware Building

→ Chapter 6: Claude Chat vs Code
Right Tool, Right Job

→ Chapter 7: The Memory Pattern
CLAUDE.md & Persistent Context

→ Getting Started: Your Path Forward

INTRODUCTION

About This Guide

> *You're a Sales Engineer. You're not a developer. You weren't hired to write code. This guide is going to make you dangerous anyway.*

This guide was built from 30 days of building real tools after hours — usually three nights a week, occasionally full weekends when I had the house to myself. Limited formal dev background, but eager to acquire more knowledge. The learning curve was steep for about a week. Then it accelerated faster than anything I'd experienced in this career.

The goal was simple: learn as much as I could about building with AI to improve how I work and live, then share those learnings with other SEs. Not to give you a toolkit — to show you what I built, what I learned, and the possibilities. So you can see what's out there and skip the mistakes I made.

This guide covers the complete stack — from first installation to working automated lab environments — with practical steps you can start immediately.

Rule of thumb: if you've explained your project to Claude more than twice in the same week, you forgot to write a CLAUDE.md.

CHAPTER 1

Why This Guide Exists

None of the projects below required writing code from scratch. Every one was built by describing what was wanted to an AI coding assistant and iterating. The AI writes the code. You tell it what to build, review what it produces, and steer.

That's what this guide teaches you to do.

My Project Examples

Project	What It Does	Time to Build	Category
SE Leaders Intelligence Briefing	Automated daily digest — aggregates industry news, competitive intel, and market data. Delivered every morning with actionable guidance on how it applies to SE teams. Runs itself.	1 day	Automation
AI Exposure Management Tool	Scans customer public-facing infrastructure for AI-related exposure risks. Exposed keys, chatbots, unauthenticated AI endpoints. Plugs directly into the Threat Profile workflow.	2 days	Security
Open Source Idea Platform	Full web app with GitHub integration, auto-generated pages, admin panel, community discussions. Built to promote idea sharing inside Sophos.	3 days	Web App
Automated Lab Manager (LabOps)	Provisions Windows VMs on Proxmox, configures environments, manages lifecycle through a web console. One command setup. Easy button for SE lab provisioning.	1 day	Infrastructure
Attack Simulation Platform	Converts real MDR cases using natural language into live, repeatable attack demos mapped to MITRE ATT&CK. Built on top of LabOps.	3 days	Demos
AI Publication + iOS Control App	Personal learning project: AI narrator running multiple websites, 11 automated workflows, Instagram pipeline, native iOS app.	5 days	Personal

"I don't know where to start."

Start with whatever annoys you. At work: a report you build manually every week, a process that takes too many clicks, a tool you wish existed. At home: a personal project you've been wanting to build, something for your family, a hobby tool.

Either path works. I built half of these examples outside of work hours before I ever applied them to SE tools. The skills transfer completely — learn by building what you care about, then bring that knowledge to Sophos.

By following this guide, you'll build real tools, automate real workflows, and understand why people keep saying "AI changes everything" — with demos to prove it.

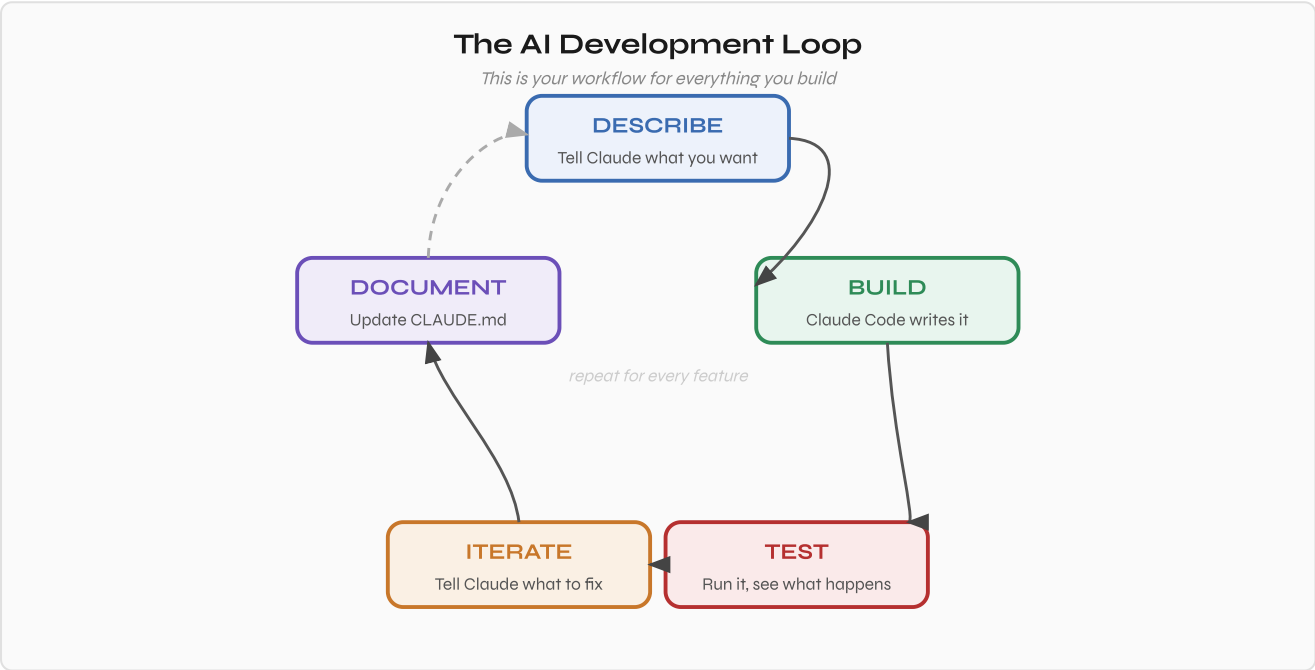
The most common first SE project: automating the weekly status report. The second most common: automating the follow-up asking why nobody filled out the status report.

CHAPTER 2

The Stack — What You Need and What It Costs

This is one SE's setup. The principles in this guide — giving AI context, documenting as you build, the plan-then-execute pattern — work regardless of which specific tools you pick. Use what clicks. Skip what doesn't.

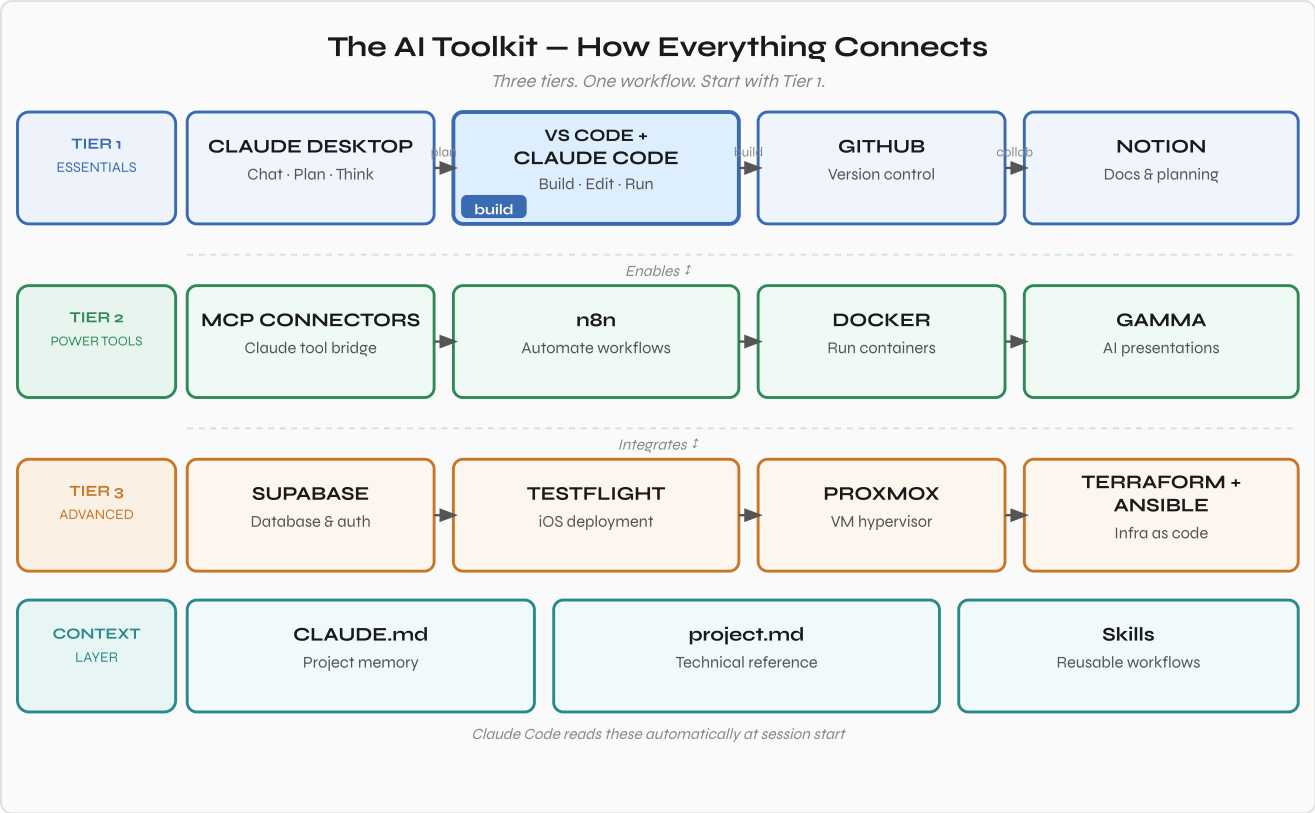
You don't need all of these on Day 1. Start with Tier 1. Add the rest as the guide introduces them.



The AI development loop — Describe, Build, Test, Iterate, Document. Repeat.

Tier 1 — The Essentials (Install These First)

Tool	What It Does	Cost	Platform
Claude Desktop	AI chat for planning, writing, analysis, brainstorming	Free / Pro $20/mo	Mac + Win
VS Code	Code editor — your workspace for everything	Free	Mac + Win
Claude Code	AI coding agent inside VS Code. Reads your files, writes code, runs commands. This is the engine.	Included with Pro or API	Mac + Win
GitHub	Version control. Your undo button, your backup, your collaboration tool.	Free	Mac + Win



The complete AI toolkit — three tiers, one workflow. Start with Tier 1.

Tier 2 — Power Tools (Add in Week 1)

Tool	What It Does	Cost	Platform
MCP Connectors	Lets Claude talk directly to Notion, n8n, databases, and more. One config file, permanent superpowers.	Free	Mac + Win
n8n	Visual workflow automation. Like Zapier but self-hosted and dramatically more powerful.	Free (self-hosted)	Mac + Win
Docker	Runs apps in containers. "It works on my machine" becomes "it works on every machine."	Free	Mac + Win
Gamma	AI presentation builder. Connected to Claude via MCP. Describe a deck, it builds it.	Free tier	Web

Tier 3 — Advanced (Week 2+)

Tool	What It Does	Cost	Platform
Supabase	Database + auth + API. A backend for your apps without needing backend experience.	Free tier	Web
TestFlight	Put an iOS app on your phone without the App Store.	Free (Apple Dev $99/yr)	Mac only
Proxmox	VM hypervisor for lab environments.	Free	Dedicated HW
Terraform + Ansible	Infrastructure as code. Describe servers in config files, machines build themselves.	Free	Mac + Win

Total cost to get started: $0–$20/month. The paid version of Claude Pro ($20/mo) is the only thing that isn't free, and it's worth every penny once you see what Claude Code can do.

Proxmox is free. Terraform is free. Ansible is free. The three weeks you'll spend learning them before realising Claude can just write the configs for you — priceless.

CHAPTER 3

Key Concepts — The Glossary You'll Actually Use

You're going to hear these terms constantly. Each one explained in plain English.

Term	Plain English Explanation
Markdown (.md files)	Simple text formatting that every tool in this stack understands. # for headers, ** for bold, - for lists. It's the common language between you, Claude, GitHub, and Notion. Takes 5 minutes to learn.
CLAUDE.md	A markdown file you put in your project folder. Claude Code reads it automatically at the start of every session. It's a briefing document: "here's what this project is, here's what we've built, here's what to do next." This single file is the difference between Claude starting confused and Claude picking up exactly where you left off. **The most important concept in this guide.**
project.md	A companion to CLAUDE.md with more detail: infrastructure specifics, API endpoints, deployment instructions, known issues. CLAUDE.md is the executive summary. project.md is the technical reference.
MCP (Model Context Protocol)	A standard that lets Claude talk to external tools directly. Instead of copy-pasting between Claude and Notion, MCP lets Claude read and write to Notion on its own. Same for n8n, databases, file systems, and dozens of other tools. Configure once, forget forever.
Skills	Packaged instructions that teach Claude how to do specific things well. Like handing a new hire a procedures manual. Claude Code comes with built-in skills, and you can create custom ones. They live in `~/.claude/skills/`.
System Prompt	Instructions you give Claude before a conversation starts. Defines its role, rules, and context. Think of it as a job description for AI.

Term	Plain English Explanation
Plan Mode (Claude Code)	A mode where Claude Code maps out everything it intends to do before writing a single line of code. Files it'll create, changes it'll make, commands it'll run. You review the plan, ask questions, adjust, then approve. Blueprint before construction.
Auto-accept / Bypass Permissions	A mode that lets Claude Code execute without pausing for permission at each step. Powerful when combined with Plan Mode. Dangerous without it.
n8n Workflows	Visual automation pipelines. Connect triggers ("when this happens") to actions ("do this") by dragging boxes and drawing lines. No code required for basic workflows.
Docker Containers	Self-contained packages that run apps identically on every machine. One command starts everything. Eliminates setup headaches.
API (Application Programming Interface)	How software talks to other software. When Claude Code creates a blog post, it's calling an API. When n8n sends an email, it's calling an API. APIs are the bridges between tools.
Git / GitHub	Version control. Every change is saved as a snapshot ("commit") you can always revert to. GitHub stores those snapshots in the cloud. It's your undo button for everything.
Terraform	Infrastructure as code. Describe what servers you want in a config file, Terraform builds them. Repeatable, consistent, shareable.
Ansible	Configuration management. Once a server exists, Ansible configures it — installs software, sets permissions, applies policies. Also defined in code files.

CHAPTER 4

The Plan-Then-Execute Pattern

> *This is the single most important workflow in the guide. Everything else builds on it.*

Claude Code has two modes that work together:

Plan Mode — Claude reads your project, analyzes your request, and produces a detailed plan: files to create, changes to make, commands to run. It executes nothing. You review, question, adjust.

Bypass Permissions — Once you've approved the plan and you're confident Claude understands the task, you enable auto-accept. Claude executes the entire plan without pausing at each step. It creates, runs, tests, iterates — all autonomously.

The Pattern

1. Describe what you want to build
2. Turn on Plan Mode — Claude maps out the full approach
3. Review: Does it understand the architecture? Is it touching files it shouldn't? Is the approach correct?
4. Adjust if needed ("don't use that library" or "start with the backend first")
5. Approve. Turn on Bypass. Let it build.
6. Come back, review output, test, iterate.

Without Plan Mode, you're approving each action one at a time — like reviewing a construction project one brick at a time instead of looking at the blueprint. With it, you see the whole plan, approve once, and let the builder work.

The Rules

Always use Plan Mode when:

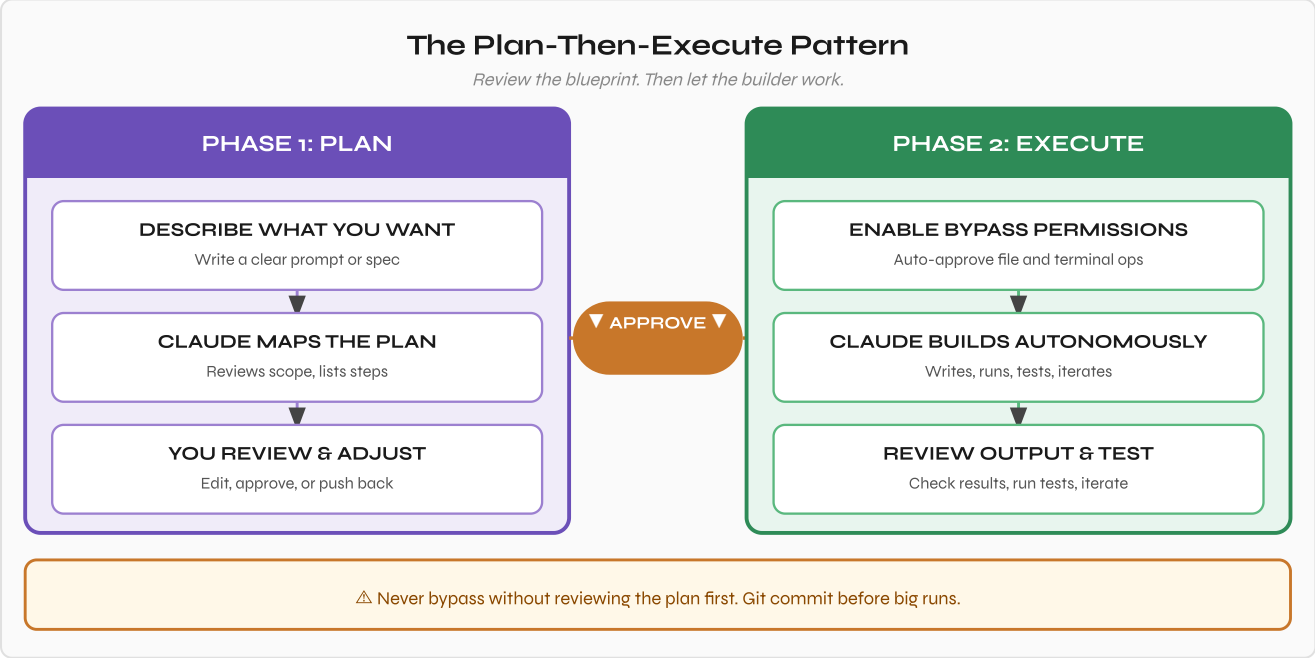
- Starting a new project
- Making architectural changes
- Working with unfamiliar libraries or frameworks
- Modifying critical infrastructure or security-related code
- You're not 100% confident Claude understands what you want

Only use Bypass Mode when:

- You've reviewed the plan and it's correct
- The task is straightforward and you trust Claude's approach
- You're iterating on something you've already built
- Time matters and you're willing to review output afterward

Never use Bypass Mode when:

- Claude hasn't produced a plan first
- You're working in production
- The task involves deleting files or data
- You don't understand what Claude is about to do



Review the blueprint. Then let the builder work.

CHAPTER 5

Dos and Don'ts — Especially for a Cybersecurity Company

You work at Sophos. Your team will immediately (and correctly) ask: "What data is being sent where?" This section answers that.

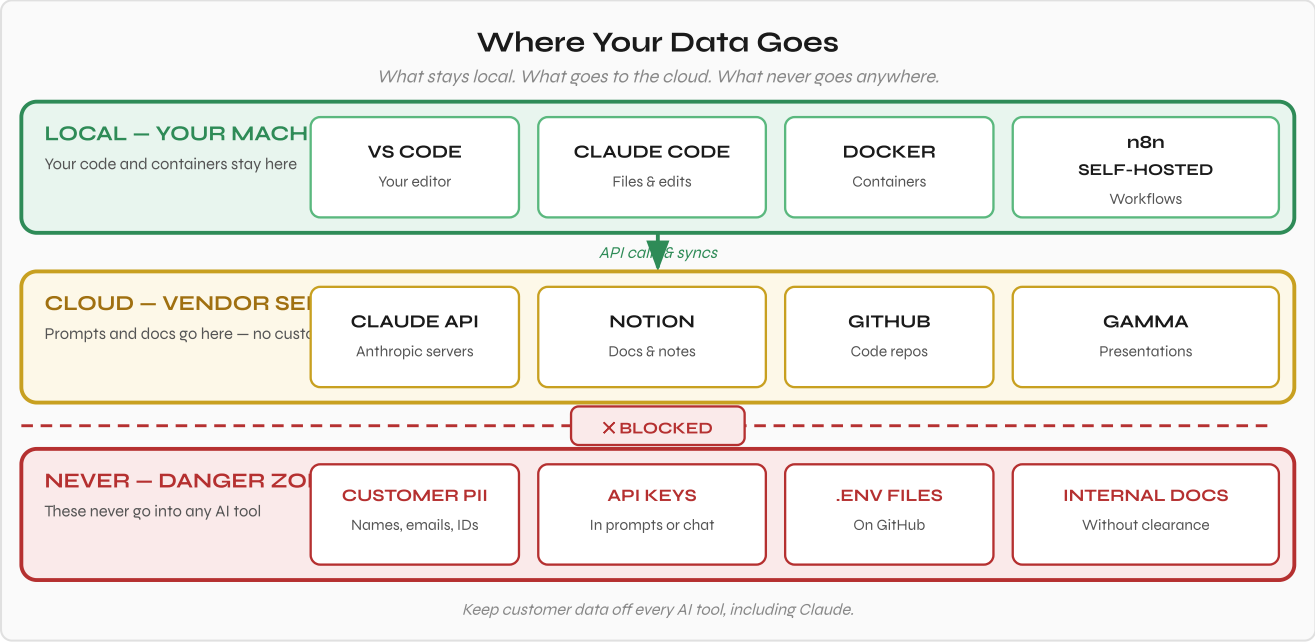
DO

- **Use CLAUDE.md in every project.** It's the single biggest productivity multiplier. No CLAUDE.md = starting from zero every session.
- **Document as you build.** Update Notion and project files after every session. Future you will be grateful.
- **"Describe, Build, Test, Iterate."** Don't try to spec everything perfectly upfront. Tell Claude what you want, see what it builds, adjust.
- **Keep secrets in .env files.** Add .env to your .gitignore. Commit a .env.example with placeholder values.
- **Ask Claude to explain.** "What does this code do?" makes you learn faster than copying blindly.
- **Use Claude to write your documentation.** It's genuinely better at structured docs than most humans. Let it.

DON'T

- **Never paste customer data into any AI tool.** No PII. No customer environment details. No case data. No exceptions.
- **Never paste credentials into a prompt.** No API keys, passwords, tokens, or secrets. If Claude needs an API key, it goes in a .env file.
- **Never commit .env files to GitHub.** This is how secrets leak. It happens to professionals constantly. Don't be that commit.
- **Never trust AI output blindly.** Review generated code before running it, especially anything touching infrastructure or customer-facing systems.
- **Never use AI-generated code in production without review.** It's a starting point, not a finished product. Test it. Read it. Understand it.

Where Your Data Goes



What stays local. What goes to the cloud. What never goes anywhere.

Tool	Where Data Lives	What to Know
Claude Desktop / Chat	Anthropic's cloud servers	Don't paste customer data or credentials. General planning and brainstorming is fine.
Claude Code	Local files + API calls to Anthropic	Your code stays local. Prompts and responses go through Anthropic's API. Same rule: no customer data.
Notion	Notion's cloud	Don't store raw credentials. Reference where to find them, never the values.
GitHub	GitHub cloud (or private repos)	Use private repos for anything work-related. .gitignore your secrets.
n8n (self-hosted)	Your machine	Stays local. Best option for workflows that touch sensitive data.
Docker	Your machine	Containers run locally. Data stays local unless you configure external storage.

CHAPTER 6

Claude Chat vs Claude Code

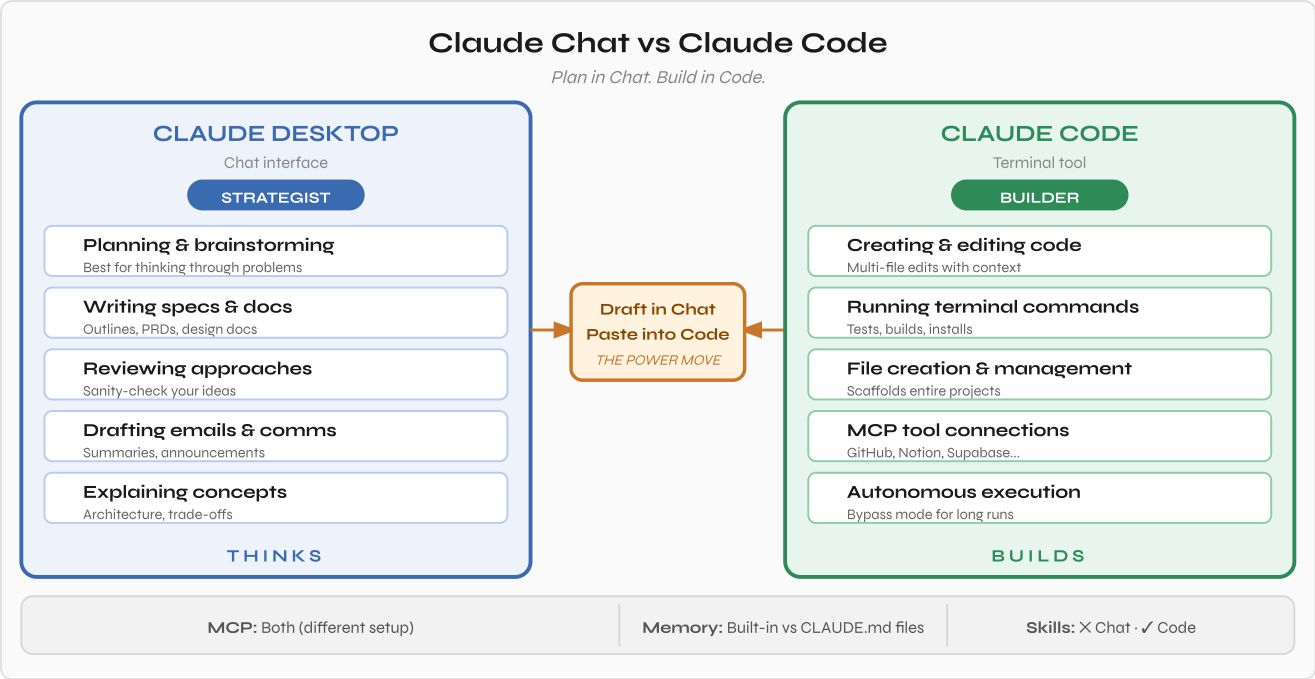
Plan in Chat. Build in Code.

One of the most common early mistakes. Takes 60 seconds to fix.

Claude Desktop (the chat app) is where you think, plan, write specs, brainstorm, review outputs, and draft non-code content. It's your **strategist**.

Claude Code (inside VS Code or terminal) is where you build. It creates files, writes code, runs commands, connects to tools via MCP, and executes. It's your **builder**.

Same intelligence. Different capabilities.



Plan in Chat. Build in Code. Same intelligence, different capabilities.

When to Use What

Task	Chat	Code	Notes
Planning and brainstorming	✓		Use Chat for thinking
Writing specs, reviewing approaches	✓		Draft in Chat first
Drafting emails, docs, strategies	✓		
Creating and editing actual code		✓	Code writes the files
Running terminal commands		✓	
File creation and testing		✓	
MCP-connected tools	Both	Both	Different config per app
Custom skills (·/.claude/skills/)	✗	✓	Skills are Code-only
Memory across sessions	Built-in memory	CLAUDE.md files	CLAUDE.md is more powerful

The power move: Draft a detailed spec in Claude Chat, then paste it into Claude Code as a prompt. Chat does the thinking. Code does the building. This is the workflow behind every project in Chapter 1.

CHAPTER 7

The Memory Pattern

CLAUDE.md and Persistent Context

> *The quality of what AI builds for you is directly proportional to the context you give it. This is not a suggestion. It's physics.*

What Goes in CLAUDE.md

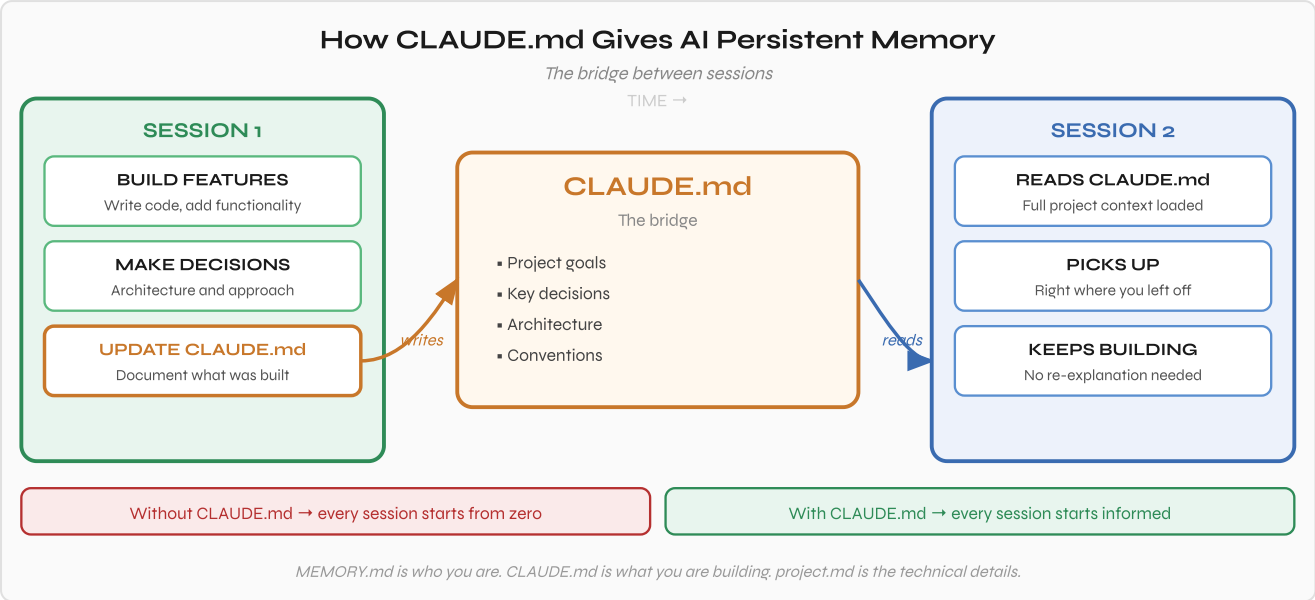
This file lives in your project root. Claude Code reads it automatically. Every time. No setup required.

```
# CLAUDE.md — [Project Name]

## Overview
[2-3 sentences: what this project is and what it does]

## Infrastructure
[What's running, where, how to access it]

## Completed Work
[What's been built so far — bullet points]

## Pending Tasks
[What's next — in priority order]

## Key Decisions
[Choices made and why — so Claude doesn't re-debate them]

## File Paths
[Important directories and files]

## Credentials
[Where to find them — NEVER the actual values]
```

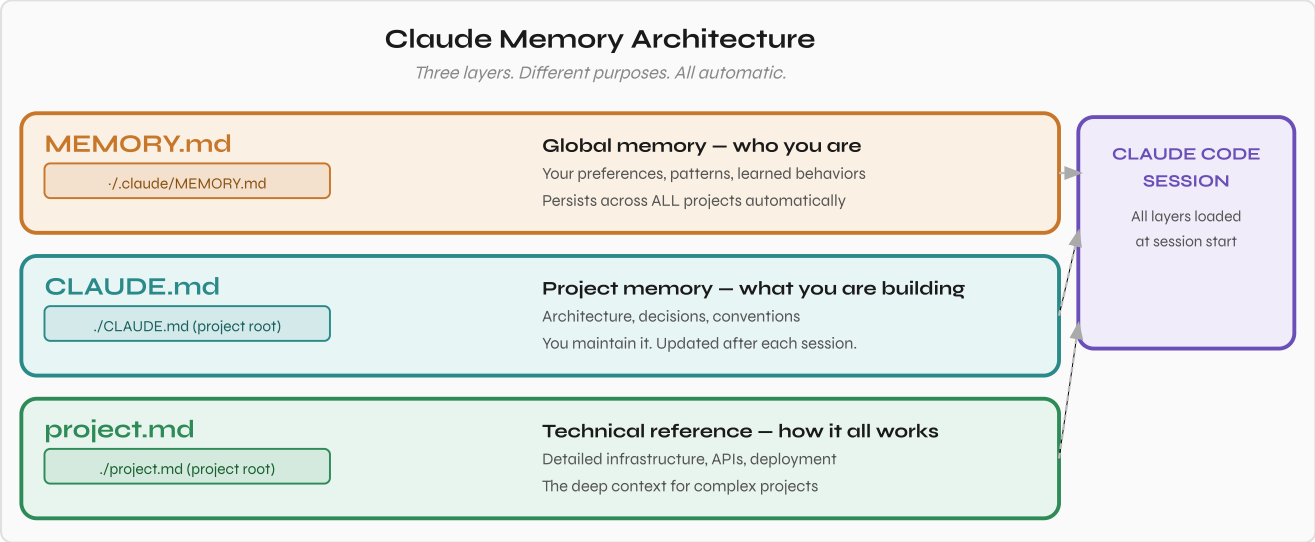
The Memory Loop

This is the pattern that makes everything work:

1. **Start a session** — Claude reads CLAUDE.md, knows exactly where things stand
2. **Build stuff** — Claude creates, modifies, tests



How context flows through Claude Code sessions via CLAUDE.md



The full memory architecture — CLAUDE.md, project.md, and session state.

Those who do this will build things in hours. Those who skip it may spend half of every session re-explaining their project.

GETTING STARTED

Your Path Forward

> *You don't need a perfect plan. You need to start building. Here's how.*

This guide has shown you the complete stack — tools, concepts, patterns, real projects. Now it's time to apply it. You don't need to master everything at once. Pick a starting point, build something small, then expand from there.

First Steps (Start Here)

Week 1: Install and Explore

1. **Set up your tools** — VS Code, Claude Desktop, Claude Code, GitHub account
2. **Build your first thing** — Ask Claude Code to create a simple HTML dashboard or automation script
3. **Create your first CLAUDE.md file** — Start the memory habit from Day 1
4. **Push to GitHub** — Get comfortable with `git add`, `git commit`, `git push`

Week 2: Add Context and Connections

1. **Set up Notion** — Create a project workspace, document what you're building
2. **Connect MCP tools** — Link Claude to Notion, enable web search, add Gamma
3. **Install Docker** — Run your first container (n8n is a good starting point)

Topics to Explore (Pick What Interests You)

Once you have the basics down, choose topics based on what you want to build. You don't need to do all of these — pick the ones that solve real problems for you.

Beginner Level

- **Automation workflows** — Use n8n to automate reports, notifications, data collection
- **Documentation tools** — Build a knowledge base, automate meeting notes, create onboarding guides
- **Data dashboards** — Visualize SE metrics, customer data, lab inventory (Sanitize sensitive data)

Intermediate Level

- **MCP integrations** — Connect Claude to Slack, Google Drive, Gmail, GitHub
- **Docker deployments** — Self-host tools (n8n, databases, web apps)
- **Web applications** — Build internal tools with Supabase backend
- **API integrations** — Connect tools that don't have native integrations

Advanced Level

- **Infrastructure as Code** — Use Terraform and Ansible to provision and configure servers
- **Lab automation** — Build LabOps-style VM provisioning and management
- **iOS/mobile apps** — Create native apps for field tools or demos
- **Custom MCP servers** — Build your own Claude connectors for proprietary systems

Where to Go for Help

Official Resources

- **Anthropic Prompt Engineering Guide** — How to write effective prompts
- **Claude Code Documentation** — Complete reference for Claude Code features
- **MCP Server Catalog** — Browse available MCP connectors

Courses & Training

- **Anthropic Academy** — Free courses on prompt engineering and AI development
- **Vanderbilt Coursera: Prompt Engineering for ChatGPT** — Transferable skills that work with Claude

Communities

- **Claude Discord** — Official Anthropic community
- **r/ClaudeAI** — Reddit community for Claude users
- **MCP GitHub Discussions** — For MCP-specific questions

The Core Pattern (Repeat This Forever)

Every project you build from here follows the same workflow:

1. **Plan** — Describe what you want in Claude Chat, refine the approach
2. **Create CLAUDE.md** — Set up the memory file before you start building
3. **Build** — Use Claude Code with Plan Mode → review → Bypass
4. **Document** — Update Notion, update CLAUDE.md, commit to GitHub
5. **Iterate** — Test, improve, add features

This pattern works whether you're building a one-hour automation or a month-long infrastructure project. The tools change. The pattern doesn't.

What Success Looks Like

After your first week:

- You've built something that works (even if it's simple)
- You have a CLAUDE.md habit
- Your projects are on GitHub
- You understand Plan-Then-Execute

After your first month:

- You've automated at least one repetitive task
- You're comfortable with Docker and n8n
- You've connected Claude to your other tools via MCP
- You've shown a colleague something you built

After three months:

- You're building tools people at Sophos actually use
- You're teaching others the patterns from this guide
- You're thinking "I could build that" when you see gaps in your workflow
- The SE team is asking you how you built things so fast

Final Advice

Start small. Your first project doesn't need to be LabOps. Build a script that saves you 10 minutes a week. Build a dashboard that answers one question you ask repeatedly. Build a tool that makes one part of your job slightly less annoying.

Document everything. Future you will thank past you. CLAUDE.md isn't optional. GitHub commits aren't optional. Notion documentation isn't optional. These aren't overhead — they're the difference between building once and rebuilding every session.